Biological Python Practice - Sequence Alignment

Martina Castellucci

2025

...

1.0 Premessa: Importazioni, Estensioni di Classi e Definizioni di Metodi in Python

Nel corso degli esercizi e degli esami svolti, uno degli aspetti più complessi riguarda la gestione dei moduli Python, l'estensione delle classi esistenti e l'implementazione di nuovi metodi in esse. Questa sezione serve come guida pratica e teorica per comprendere cosa significa:

- **import vs from ... import ...** e quando usarli;
- **come estendere una classe esistente** con ereditarietà;
- **come implementare nuovi metodi** in una classe estesa;
- **come definire correttamente i metodi** (`def`) e lo `__str__()` per stampare oggetti custom;
- il ruolo di `super()` nella chiamata del costruttore base.

1.1 import vs from ... import ...

- `import modulo` → si accede con il nome completo: `modulo.Classe` o `modulo.funzione()`
- `from modulo import Classe` → si accede direttamente: `Classe()`

Esempio tratto dagli esercizi:

```python
# modo 1
import exammm
obj = exammm.SubstitutionMatrix("PAM250.txt")

# modo 2 (pi  diretto)
from exammm import SubstitutionMatrix
obj = SubstitutionMatrix("PAM250.txt")
```

1.2 Estendere una classe esistente (inheritance)

Quando si vuole aggiungere o modificare il comportamento di una classe esistente (es. `SequencePair`), si può estendere:

```python
from exammm import SequencePair
class Seq(SequencePair):
    def __init__(self, seq1, seq2):
        super().__init__(seq1, seq2)  # chiama il costruttore originale
```

1.3 Aggiungere un nuovo metodo a una classe esistente

Un metodo si aggiunge definendo una nuova funzione all'interno della classe:

```
class Seq(SequencePair):
    def alignment(self, matrix, gap, algorithm):
        # codice personalizzato
        return result
```

Questa tecnica è stata usata nell'esercizio in cui si chiedeva di unificare `Needleman-Wunsch` e `Smith-Waterman` in un unico metodo.

1.4 Metodo speciale __str__() per la stampa

Serve per rendere stampabile un oggetto:

```
class AlignedSequences:
    def __str__(self):
        return f"{self.seq1}\n{self.seq2}\nScore: {self.score}"
```

1.5 Uso di super()

`super()` serve per richiamare metodi (tipicamente il costruttore `__init__`) della classe base.

Esempio nel codice di estensione:

```
class Seq(SequencePair):
    def __init__(self, seq1, seq2):
        super().__init__(seq1, seq2)
```

1.6 Applicazione concreta: soggetto d'esame

Nel tuo esame con `smith_waterman` e `threshold`, hai usato:

- `from input import SeqPair, SubstitutionMatrix, AlnSeq`
- Estensione della classe con `class Seq(SeqPair): ...`
- Aggiunta del metodo `smith_waterman(...)` che restituisce `list[AlnSeq]`

Tutti questi elementi fanno parte della pratica di oggetti in Python (OOP) e sono fondamentali per la risoluzione strutturata degli esercizi d'esame.

1. Introduction

In this practice, we implement biological sequence alignment in Python using object-oriented programming. The goal is to compare two sequences using dynamic programming with substitution matrices and gap penalties.

We cover both global alignment (Needleman-Wunsch) and local alignment (Smith-Waterman), comparing their implementation and behavior, and including a threshold-based variant for selecting biologically relevant local alignments.

2. Substitution Matrix (TTM) Loader

```python
#use the file TTM.txt per rappresentare una matrice di sostituzione
#leggere una matrice di sostituzione da un file chiamato TTM.txt e accedere ai valori di quella matrice utilizzando due nucleotidi come chiavi.
class Substitutionmatrix:
    def __init__(self, filename):
        '''
        dictionary of dictionaries
            keys: nucleotide
            values: dictionary
                keys: nucleotide
                values: numbers
        '''
        # Apertura del file per la lettura
        with open(filename) as reader:
            # Leggi la prima riga per ottenere i nucleotidi (colonne e righe della matrice)
            line = reader.readline()
            # Rimuovi gli spazi iniziali e finali e separa i nucleotidi in una lista
            characters = line.strip().split()

            # Inizializza il dizionario principale che conterr  la matrice di sostituzione
            self.values = {}

            # Crea una voce nel dizionario per ogni nucleotide (come chiavi)
            #Le chiavi del dizionario principale sono nucleotidi(come 'A', 'T', 'C', 'G').
            #I valori per ogni chiave sono altri dizionari, in cui le chiavi sono sempre i
            #nucleotidi e i valori sono numeri che rappresentano i valori di sostituzione tra i nucleotidi.
            for char in characters:
                self.values[char] = {}

            # Per ogni riga successiva nel file, che contiene i valori della matrice
            for line in reader:
                # Dividi la riga in base agli spazi
                splitted_line = line.strip().split()
                # Il primo elemento della riga    il nucleotide di riferimento (char1)
                char1 = splitted_line[0]

                # Soluzione 1: utilizzo di un indice per associare i valori
                # Itera sui valori della riga a partire dal secondo elemento
                for i in range(1, len(splitted_line)):
                    # Popola il dizionario con i valori
                    self.values[char1][characters[i - 1]] = float(splitted_line[i])

```

```python
                # Soluzione 2: usando la funzione zip per associare direttamente i nucleotidi ai valori
                # Questa parte    commentata, ma    un'alternativa
                # for char2, val in zip(characters, splitted_line[1:]):
                # self.values[char1][char2] = val

    # Metodo per ottenere un valore dalla matrice usando due nucleotidi come chiavi
    def __getitem__(self, key):
        key1, key2 = key
        # Restituisce il valore corrispondente nella matrice (es. "A", "T")
        return self.values[key1][key2]


# Verifica se il file    eseguito direttamente
if __name__ == "__main__":
    # Creazione dell'oggetto della classe SubstitutionMatrix, passando il nome del file
    matrix = Substitutionmatrix("TTM.txt")

    # Stampa il valore nella matrice per la sostituzione tra 'A' e 'T'
    print(matrix["A", "T"])
    print(matrix["A", "C"])
    print(matrix["A", "G"])
    print(matrix["T", "A"])
    print(matrix["T", "C"])


#     A  T  C  G
# A   2 -1 -1  0
# T  -1  2  0 -1
# C  -1  0  2 -1
# G   0 -1 -1  2
```

Listing 1: Substitutionmatrix.py - DNA substitution matrix loader

Explanation: This class reads a substitution matrix (like PAM250 or TTM) into a dictionary-of-dictionaries structure, allowing quick access to substitution scores using tuple keys.

```python
class SubstitutionMatrix:
    def __init__(self, filename):
        '''
        Dictionary-based substitution matrix
            - Keys: tuples of nucleotides (char1, char2)
            - Values: substitution scores
        '''
        with open(filename) as reader:
            # Leggi la prima riga e crea una lista con i caratteri
            line = reader.readline()
            characters = line.strip().split()

            # Inizializza il dizionario
            self.values = {}

            # Per ogni riga successiva nel file, che contiene i valori della matrice
            for line in reader:
                splitted_line = line.strip().split()
```

```python
                char1 = splitted_line[0]

                # Associa i valori ai nucleotidi usando le tuple come chiave
                for i in range(1, len(splitted_line)):
                    char2 = characters[i - 1]
                    value = float(splitted_line[i])
                    self.values[(char1, char2)] = value

    def __getitem__(self, key):
        return self.values[key]  # La chiave    ora una tupla (char1, char2)


# Verifica se il file    eseguito direttamente
if __name__ == "__main__":
    matrix = SubstitutionMatrix("TTM.txt")

    print(matrix["A", "T"])
    print(matrix["A", "C"])
    print(matrix["A", "G"])
    print(matrix["T", "A"])
    print(matrix["T", "C"])
```

Listing 2: SubstitutionMatrix with tuple

3. Needleman-Wunsch Algorithm (Global Alignment)

```python
#Needleman-Wunsch Algorithm (Global Alignment)
#submat    un oggetto della classe SubstitutionMatrix, che contiene la matrice di sostituzione letta da un file esterno (come TTM.txt).
#Viene usato nell'algoritmo di Needleman-Wunsch per calcolare i punteggi di allineamento tra le due sequenze, utilizzando i valori della matrice di sostituzione per determinare la "corrispondenza" tra i nucleotidi o amminoacidi.

from matrices import Substitutionmatrix
class SeqPair:
    def __init__(self, seq1, seq2):
        '''
                Il costruttore della classe. Riceve due sequenze (seq1, seq2) come input
                e le memorizza come attributi dell'oggetto.
        '''
        self.seq1 = seq1
        self.seq2 = seq2

    def _create_matrices(self, matrix, gap):
        '''
                Crea le matrici F e T:
                - F: matrice dei punteggi di allineamento
                - T: matrice che memorizza le direzioni del backtracking ('u' = up, 'l' = left, 'd' = diagonal)

                matrix: la matrice di sostituzione
                gap: penalizzazione per l'inserimento di un gap
        '''
```

```python
        # Crea matrici vuote F e T (F contiene i punteggi, T le direzioni)
        F = []  # matrix that contains all the values at the end of the execution
        T = []  # only store the traceback

        # Inizializza le matrici con dimensioni adeguate (len(seq2)+1, len(seq1)+1)
        for row in range(len(self.seq2) + 1):  #+1 gap
            F.append([])
            T.append([])  # Aggiungi una nuova riga a T (necessario)
            for col in range(len(self.seq1) + 1):
                F[row].append(0)  # Corrected index assignment # Inizializza F con 0
                T[row].append('') # Inizializza T con stringhe vuote

        # Inizializzazione della prima riga (per i gap orizzontali)
        for col in range(1,len(self.seq1) + 1):
            F[0][col]=gap*col  # Punteggio basato sul gap
            T[0][col]="l"  # Direzione 'l' per left

        # Inizializzazione della prima colonna (per i gap verticali)
        for row in range(1,len(self.seq2) + 1):
            F[row][0]=F[row-1][0] + gap  # Punteggio basato sul gap
            T[row][0]="u" # Direzione 'u' per up

        # Riempie il resto delle matrici F e T
        for row in range(1, len(self.seq2) + 1):
            for col in range(1, len(self.seq1) + 1):
                '''
                    Calcola i valori nelle matrici F e T per ogni cella
                    Versione 3 con Tuples:
                    Usa una tupla per associare i punteggi e la direzione:
                    - Il punteggio per il movimento verticale (up, 'u')
                    - Il punteggio per il movimento orizzontale (left, 'l')
                    - Il punteggio per il movimento diagonale (diagonal, 'd')
                '''

                top=F[row-1][col]+ gap, 3, 'u'  # Punteggio e direzione 'u'
                left=F[row][col-1] + gap, 2, 'l'  # Punteggio e direzione 'l'
                diagonal=F[row-1][col-1] + matrix[self.seq2[row-1], self.seq1[col-1]], 1, 'd'  # Punteggio e direzione 'd'

                # Prende il massimo tra i punteggi e assegna il valore e la direzione corretti
                F[row][col], _, T[row][col] = max(top, left, diagonal)

        return F, T

    def needleman_wunsch(self, matrix, gap):
        '''
                Esegue l'algoritmo Needleman-Wunsch per l'allineamento globale delle due sequenze.
```

```
                matrix: la matrice di sostituzione gap: penalizzazione per i gap
        '''

        # Crea le matrici F e T utilizzando il metodo _create_matrices
        F,T= self._create_matrices(matrix, gap)

        # Inizializza le sequenze allineate
        aln1= ""
        aln2= ""

        # Inizia il backtracking dal basso a destra (angolo inferiore destro)
        row=len(self.seq2)
        col=len(self.seq1)

        # Ciclo fino a raggiungere l'angolo in alto a sinistra
        while row!=0 or col!=0:
            # Se la direzione    diagonale ('d'), allinea entrambi i caratteri
            if T[row][col]=='d':
                aln1=aln1+self.seq1[col-1]
                aln2=aln2+self.seq2[row-1]
                row = row - 1
                col = col - 1
            # Se la direzione    verso sinistra ('l'), aggiungi un gap alla sequenza 2
            elif T[row][col]=='l':
                aln1=aln1+self.seq1[col-1]
                aln2=aln2+"-"
                col=col - 1
            # Se la direzione    verso l'alto ('u'), aggiungi un gap alla sequenza 1
            else: # T[i][j] == 'u'
                aln2=aln2+self.seq2[row-1]
                aln1=aln1+"-"
                row=row-1
        # Restituisce le due sequenze allineate e il punteggio finale
        return aln1, aln2, F[-1][-1]


# Esegui l'algoritmo Needleman-Wunsch se il file viene eseguito come script
if __name__ == '__main__':
    # Crea un oggetto SeqPair con le sequenze da allineare
    myobj = SeqPair("TCA", "TA")
    # Carica la matrice di sostituzione da un file chiamato TTM.txt
    submat = Substitutionmatrix("../TTM.txt")
    # Esegui l'allineamento Needleman-Wunsch e ottieni le sequenze allineate e il punteggio finale
    aln1, aln2, score =myobj.needleman_wunsch(submat, -2)
    # Stampa i risultati
    print(aln1)
    print(aln2)
    print("Score: ", score)
#Come funziona l'algoritmo con questa matrice:
#Match (Corrispondenza): I nucleotidi uguali ricevono un punteggio positivo (ad esempio, A-A o T-T hanno un punteggio di +2).
```

```
#Mismatch (Sostituzione non corrispondente): I nucleotidi diversi ricevono un punteggio negativo (ad esempio, A-T ha un punteggio di -1).
#Gap penalty: Ogni volta che viene inserito un gap, viene applicata una penalizzazione di -2.

#output:
#ACT
#A-T
#Score:  2.0
```

Listing 3: Global alignment with traceback directions

Theory: Needleman-Wunsch is a dynamic programming algorithm for textitglobal alignment, meaning both sequences are aligned from start to end. Gap penalties are applied even at the beginning or end.

4. Smith-Waterman Algorithm (Local Alignment)

```
#In questo codice viene implementato l'algoritmo Smith-Waterman per l'allineamento locale delle sequenze, che    una variazione dell'algoritmo di Needleman-Wunsch.
#Mentre Needleman-Wunsch esegue un allineamento globale tra due sequenze, Smith-Waterman cerca di trovare l'allineamento ottimale tra due sequenze che possono essere parzialmente allineate (cio  con un segmento allineato, senza bisogno che le sequenze siano completamente allineate).
#Per farlo, si utilizza una matrice di sostituzione e una penalizzazione per i gap.

#La classe SeqPair estende la classe SeqPair definita nel modulo Substitutionmatrix
# per aggiungere nuovi metodi necessari per l'algoritmo Smith-Waterman
from manfredi import Substitutionmatrix, matrices

class SeqPair(Substitutionmatrix.SeqPair):  # Use inheritance to extend the original class with new methods
    def __init__(self, seq1, seq2):
        ''' Questo costruttore chiama il costruttore della classe originale '''
        super().__init__(seq1, seq2)  # Inizializza le sequenze utilizzando il costruttore della classe base

    # Metodo per creare le matrici F e T utilizzate per l'algoritmo Smith-Waterman
    def _create_local_matrices(self, matrix, gap):
        # Crea le matrici F e T (F contiene i punteggi di allineamento, T contiene le direzioni di backtracking)
        F = []  # Matrice dei punteggi
        T = []  # Matrice delle direzioni

        # Inizializza le matrici con dimensioni adeguate
        for row in range(len(self.seq2) + 1):
            F.append([0] * (len(self.seq1) + 1)) # Inizializza la matrice F con 0
            T.append(['''] * (len(self.seq1) + 1)) # Inizializza la matrice T con stringhe vuote
```

```python

        # Riempie il resto delle matrici F e T
        for row in range(1, len(self.seq2) + 1):
            for col in range(1, len(self.seq1) + 1):
                # Calcola i valori per ogni cella (top, left, diagonal, zero)
                '''
                # Version 1 with if elif elif else
                top = F[row-1][col] + gap
                left = F[row][col-1] + gap
                diagonal = F[row-1][col-1] + matrix[self.seq1[col-1], self.seq2[row-1]]

                F[i][j] = max(top, left, diagonal, 0)

                if F[row][col] == 0: # First choice in case of parity pass
                elif F[row][col] == diagonal:
                    T[row][col] = 'd'
                elif F[row][col] == top:
                    T[row][col] = 'u'
                else: #F[row][col] == left:
                    T[row][col] = 'l'
                '''

                # Version 3 with tuples
                top = F[row - 1][col] + gap, 3, 'u'  # Punteggio se il movimento    verso l'alto (up)
                left = F[row][col - 1] + gap, 2, 'l' # Punteggio se il movimento    verso sinistra (left)
                diagonal = F[row - 1][col - 1] + matrix[self.seq2[row - 1], self.seq1[col - 1]], 1, 'd' # Punteggio per il movimento diagonale (diagonal)
                zero = 0, 4, '' # Prima scelta in caso di parit  (se il punteggio    0)
                F[row][col], _, T[row][col] = max(top, left, diagonal, zero) # Calcola il massimo tra i punteggi e aggiorna F e T

        return F, T #restituisce le matrici riempite

    # Metodo che implementa l'algoritmo Smith-Waterman per l'allineamento locale
    def smith_waterman(self, matrix, gap):
        # Crea le matrici F e T utilizzando il metodo _create_local_matrices
        F, T = self._create_local_matrices(matrix, gap)
        aln1 = "" # Sequenza allineata 1
        aln2 = "" # Sequenza allineata 2

        # Trova la posizione di inizio per il backtracking (il massimo punteggio in F)
        max_score = -1
        for row in range(len(self.seq2) + 1):
            for col in range(len(self.seq1) + 1):
                if F[row][col] > max_score:
                    max_score = F[row][col] # Aggiorna il punteggio massimo
```

```python
                    start_row = row # Memorizza la posizione di inizio (riga)
                    start_col = col # Memorizza la posizione di inizio (colonna)

        row = start_row # Imposta la riga di partenza per il backtracking
        col = start_col  #Imposta la colonna di partenza per il backtracking

        # Ciclo di backtracking finch  non raggiungiamo una direzione vuota ('')
        while T[row][col] != '':
            if T[row][col] == 'd': # Se la direzione    diagonale ('d')
                aln1 = self.seq1[col - 1] + aln1 # Aggiungi il carattere da seq1 all'allineamento
                aln2 = self.seq2[row - 1] + aln2 # Aggiungi il carattere da seq2 all'allineamento
                row = row - 1 # Muovi verso l'alto
                col = col - 1 # Muovi verso sinistra
            elif T[row][col] == 'l': # Se la direzione    verso sinistra ('l')
                aln1 = self.seq1[col - 1] + aln1 # Aggiungi il carattere da seq1 all'allineamento
                aln2 = "-" + aln2 # Aggiungi un gap a seq2
                col = col - 1 # Muovi verso sinistra
            else:  # se la direzione    verso l'alto ('u')
                aln1 = "-" + aln1 # Aggiungi un gap a seq1
                aln2 = self.seq2[row - 1] + aln2  #Aggiungi il carattere da seq2 all'allineamento
                row = row - 1 # Muovi verso l'alto

        # Restituisce le due sequenze allineate e il punteggio massimo
        return aln1, aln2, max_score

# Codice principale
if __name__ == "__main__":
    # Crea un oggetto SeqPair con le sequenze da allineare
    myobj = SeqPair("TGA", "GA")  # Inizializza l'oggetto con le sequenze "TGA" e "GA"
    # Stampa le sequenze per verificare l'input
    #print(myobj.seq1, myobj.seq2)
    # Carica la matrice di sostituzione dal file "TTM.txt"
    submat = matrices.Substitutionmatrix("../exam manfredi/2024-02/PAM250.txt")
    # Esegui l'allineamento Smith-Waterman e ottieni le sequenze allineate e il punteggio finale
    aln1, aln2, score = myobj.smith_waterman(submat, -2)
    print(aln1)
    print(aln2)
    print("Score: ", score)

#output:
#GA
#GA
#Score:  4.0
```

Listing 4: smithwaterman.py - Local alignment implementation

Theory: Smith-Waterman performs
textitlocal alignment: it aligns the most similar region between two sequences. The matrix allows scores to reset to 0 to discard poorly matching regions.

5. Differences Between Global and Local Alignment

- **Initialization:** NW fills first row/column with gap penalties, SW fills with zeros.
- **Filling:** NW uses max of 3 options, SW includes a fourth option: 0.
- **Traceback:** NW starts bottom-right, SW starts from highest-scoring cell.
- **Use case:** NW = similar sequences end-to-end. SW = partial alignments, domains, motifs.

6. Final Alignment Pipeline Summary

Both alignment algorithms follow the same pipeline structure:

1. **Initialization**: create matrices `F` (scores) and `T` (traceback directions).
2. **Filling the matrix**: use substitution matrix values and gap penalties.
3. **Traceback**: reconstruct aligned sequences using directions in `T`.
4. **Return**: aligned sequences and final score.

Main differences:

- NW initializes with gap penalties, SW with zeros.
- SW resets values to zero when scoring is negative.
- NW always traces from bottom-right, SW from highest cell.

7. Programming Concepts Explanation

- **Class**: A template for creating objects with attributes and methods.
- **self**: Refers to the current instance of the class.
- **def**: Defines a function or method.
- **zip()**: Pairs elements from two iterables.
- **range()**: Generates a sequence of numbers.
- **tuple**: Immutable pair or group of values, like `(score, priority, direction)`.
- **Dictionary**: A key-value store used to hold matrices, e.g. `values['A']['T']`.

8. Matrix and Direction Priorities

- **Matrix F**: stores optimal scores.
- **Matrix T**: stores direction of best move ('u', 'l', 'd').
- **Priority**: Used in tuples to break ties: $diagonal > left > up > zero$.
- **Rows/Columns**: Rows = sequence 2 (vertical), Columns = sequence 1 (horizontal).

9. Example Matrix (Scoring)

```
     A  T  C  G
A |  2 -1 -1  0
T | -1  2  0 -1
C | -1  0  2 -1
G |  0 -1 -1  2
```

10. Substitution Matrices in Biology

- **PAM**: based on evolutionary mutation rates over time.
- **BLOSUM**: based on blocks of aligned protein sequences.
- Used for protein scoring: higher value = more likely substitution.

11. Distance vs Similarity

- **Distance**: number of operations to convert one sequence into another.
- **Similarity**: biological relevance of matches (e.g. high for similar amino acids).

12. Matches, Mismatches and Gaps

- **Match**: characters aligned and equal → positive score.
- **Mismatch**: characters aligned and different → negative score.
- **Gap**: alignment with '-' → gap penalty (e.g. -2 or -5).

13. Classes and Bioinformatics: Practical Connections

In Python, using classes to model biological data helps structure both logic and data handling. Here's how each programming concept links directly to bioinformatics tasks:

Real-world mapping of OOP concepts

- **Class** → represents an abstraction like a sequence pair or substitution matrix.
- **Object** → an instance of a matrix (e.g., PAM250), or a specific alignment task.
- **Attributes** → sequence strings (`seq1`, `seq2`), matrix dictionary (`values`).
- **Methods** → logic to compute alignment (`needleman_wunsch()`, `smith_waterman()`).
- **Encapsulation** → internal methods like `_create_matrices()` are hidden from external code.

Why classes improve biological alignment code

- **Reusability**: once a class like `SeqPair` is defined, it can be used in both NW and SW.
- **Extensibility**: the alignment behavior can be changed by overriding a method.
- **Interoperability**: can be integrated with libraries like Biopython.

Python features applied in biology code

- `zip()` → used to combine headers and values from substitution matrix files.
- `range()` → drives loops through rows and columns of alignment matrices.
- `tuples` → used to track score and direction at once: `(score, priority, direction)`.
- `dictionaries` → matrix["A"]["T"] or matrix["G", "C"]: flexible and fast access.

From concept to code: biological meaning of matrices

- Each cell `F[i][j]` in the score matrix represents the best alignment score between prefixes: `seq2[:i]` and `seq1[:j]`.
- `T[i][j]` contains the direction of the best step to reconstruct the alignment: `d` (diagonal), `l` (left), `u` (up).

Substitution matrices: PAM vs BLOSUM

- **PAM**: derived from evolutionary models (e.g., PAM250 simulates 250 accepted mutations).
- **BLOSUM**: derived from observed substitutions in aligned protein blocks (e.g., BLOSUM62).
- PAM is extrapolative; BLOSUM is observational.

Distance vs Similarity: which metric?

- **Distance metrics** (like Levenshtein): measure "how far" sequences are.
- **Similarity scores** (like NW/SW): measure "how closely" sequences match biologically.
- Most alignment tools use similarity, with specific substitution and gap scores.

Match, mismatch and gap: biological scoring

- **Match** → +1 to +2 (amino acids or nucleotides conserved).
- **Mismatch** → -1 to -4 (functional divergence).
- **Gap penalty** → -2 to -10 (insertion/deletion events).

In summary: classes enable modular and reusable code to perform biologically meaningful sequence analysis. The structure of the code reflects the structure of biological information.

EXTRA: Interpreting Row and Column Indexes in Alignment Matrices

In the alignment matrix, we define:

- `row` → represents position in **seq2**, the vertical sequence (written on the left).
- `col` → represents position in **seq1**, the horizontal sequence (written on top).

This structure is visualized as:

```
    -  A  C  G  T   (seq1 → horizontal → col)
  ----------------
- |  0  -2 -4 -6
A | -2
G | -4
T | -6
(seq2 ↓ vertical → row)
```

Placement of Gaps

- A **gap in seq1** corresponds to a move **downward** in the matrix: `T[row][col] = 'u'`
- A **gap in seq2** corresponds to a move **leftward** in the matrix: `T[row][col] = 'l'`
- A **match or mismatch** is a move **diagonal**: `T[row][col] = 'd'`

This logic is consistent across both NW and SW algorithms, and is crucial for reconstructing the alignment.

14. Exam Exercise: Extended SubstitutionMatrix and SequencePair (Smith-Waterman)

Task Summary

You were asked to extend the functionality of the `SubstitutionMatrix` and `SequencePair` classes in order to perform local sequence alignment using the Smith-Waterman algorithm, while adding support for gap penalties and file input.

Implemented Changes

- **SubstitutionMatrix**: The class was extended to override the `__getitem__` method. If either character in the queried pair is a gap (`'-'`), it returns a fixed penalty of `-5`. Otherwise, it returns the original substitution score.
- **SequencePair**: The class was extended to read the two sequences from a `.fasta` file. The `_create_matrices()` method was reimplemented to fill matrices `F` and `T` as required by Smith-Waterman. Standard traceback logic was used to recover the optimal local alignment.

Main Execution Test

```python
import exercise as input

class SubstitutionMatrix(input.SubstitutionMatrix):
    def __init__(self, filename):
        super().__init__(filename)

    def __getitem__(self, key):
        if '-' in key:
            return -5
        else:
            return self.values[key]


class SequencePair(input.SequencePair):
```

```python
    def __init__(self, filename):
        with open(filename, 'r') as sequences:
            sequences.readline()
            self.seq1 = sequences.readline().strip()
            sequences.readline()
            self.seq2 = sequences.readline().strip()

    def _create_matrices(self, matrix):
        # crea la matrice
        F = []
        T = []
        # prima riga e prima colonna
        for i in range(len(self.seq2) + 1):
            F.append([0] * (len(self.seq1) + 1))
            T.append(([''']) * (len(self.seq1) + 1))
        # restante parte della matrice
        for i in range(1, len(self.seq2) + 1):
            for j in range(1, len(self.seq1) + 1):
                top = F[i - 1][j] + matrix[self.seq1[j - 1], '-'], 'u'
                left = F[i][j - 1] + matrix['-', self.seq2[i - 1]], 'l'
                diagonal = F[i - 1][j - 1] + matrix[self.seq1[j - 1],
                    self.seq2[i - 1]], 'd'
                zero = 0, ''
                F[i][j], T[i][j] = max(top, left, diagonal, zero)
        return (F, T)
if __name__ == "__main__":
    myobj = SequencePair("input.fasta")
    matrix = SubstitutionMatrix("PAM250.txt")
    print(myobj.smith_waterman(matrix))
```

This script reads the PAM250 substitution matrix and the input sequences, computes the local alignment, and prints the aligned sequences along with the alignment score.

15. Exam Exercise: General Alignment Method Supporting NW and SW

Task Summary

You were asked to implement a general alignment method that:

- Accepts a substitution matrix, a gap penalty, and a string specifying the algorithm ('nw' or 'sw').
- Returns an `AlignedSequences` object.
- Supports both global (Needleman-Wunsch) and local (Smith-Waterman) alignment.

Implemented Method

The new class `Seq` extends `SequencePair` and implements the method `Alignment(...)`.

```python
def Alignment(self, matrix, gap, algorithm):
    F, T = self._create_matrices(matrix, gap, algorithm)

    if algorithm == 'nw':
        i, j = len(self.seq1), len(self.seq2)
    else:
        max_val = -1
```

```
        for i in range(len(self.seq1)+1):
            for j in range(len(self.seq2)+1):
                if F[i][j] > max_val:
                    max_val, start_i, start_j = F[i][j], i, j
        i, j = start_i, start_j

    aln1, aln2 = "", ""
    while i != 0 or j != 0:
        if algorithm == "sw" and T[i][j] == '':
            break
        if T[i][j] == 'l':
            aln1 += '-'
            aln2 += self.seq2[j-1]
            j -= 1
        elif T[i][j] == 'u':
            aln1 += self.seq1[i-1]
            aln2 += '-'
            i -= 1
        else:
            aln1 += self.seq1[i-1]
            aln2 += self.seq2[j-1]
            i -= 1
            j -= 1

    return AlignedSequences(aln1[::-1], aln2[::-1], F[-1][-1] if
        algorithm == 'nw' else max_val)
```

Main Execution Test

```

my_obj = Seq("AAAAACCCC", "RRRRCCCC")
submat = SubstitutionMatrix("PAM250.txt")
result_nw = my_obj.Alignment(submat, -5, 'nw')
result_sw = my_obj.Alignment(submat, -5, 'sw')
print(result_nw)
print(result_sw)
```

Rationale for the Implementation

- The solution reuses the `create`$_{matricesmethodfromthebaseclass.ItseparatesNWandSWlogiconlywhere}$
- The traceback logic is standard and supports both algorithms.
- The returned `AlignedSequences` object formats the result for easy visual interpretation.

This implementation is modular, concise, and fulfills all exam requirements correctly.
...(previous content unchanged)...

18. Threshold-Based Alignment Execution

Objective

This task reused the previous generalized alignment class `Seq` to compute both global and local alignments for two specific sequences and print them to the terminal. This test was designed to validate alignment scoring and gap handling when paired with the `PAM250.txt` substitution matrix.

Code Overview

```
#Write a script where you:
#1.     Import the module exam_test.py (the file will be provided during the exam)
#2.     Extend the definition of the class SeqPair defining a new method
#a.     Name: smith_waterman
#b.     Parameters: a matrix (type SubstitutionMatrix), a gap score (type int) and a threshold (type int)
#c.     Output: a list of objects, each of type AlnSeq. Each object in the list represents a possible local alignment. The list should contain all the alignments found by the Smith and Waterman algorithm with a score equal to or greater than the threshold parameter.
#3.     Write a test main that reads a threshold parameter from the command line and adopts the defined classes to print to the terminal all the local alignments of two sequences.
#a.     First sequence: "IFYNAVWSA"
#b.     Second sequence: "IFYNAVWSAWWWVYYHAIWPG"
#c.     Substitution matrix: PAM250 (a file that can be read by the class SubstitutionMatrix will be provided during the exam)
#d.     Gap penalty: 5
from sys import argv
from input import SeqPair, SubstitutionMatrix, AlnSeq

class SubstitutionMatrix(SubstitutionMatrix):
    def __init__(self, filename):
        with open(filename) as reader:
            characters = reader.readline().strip().split()
            self.values = {}
            for line in reader:
                line = line.strip().split()
                self.values[line[0]] = {}
                for char in range(1, len(line)):
                    self.values[line[0]][characters[char - 1]] = float(line[char])

    def __getitem__(self, key):
        key1, key2 = key
        return self.values[key1][key2]


class AlnSeq(AlnSeq):
    def __init__(self, s1, s2, score):
        super().__init__(s1, s2, score)

    def __str__(self):
        return self.s1 + '\n' + self.s2 + '\n' + 'Score: ' + str(self.score)


class Seq(SeqPair):
    def __init__(self, s1, s2, threshold):
        super().__init__(s1, s2)
        self.threshold = threshold

    def smith_waterman(self, matrix, gap):
        F, T = self._local_matrices(matrix, gap)
        alignments = []
```

```python
        start_row,start_col=0,0
        for row in range(1, len(self.s1) + 1):
            for col in range(1, len(self.s2) + 1):
                if F[row][col] >= self.threshold:
                    start_row, start_col = row, col
                    aln1, aln2 = "", ""
                    score = F[row][col]
                    while T[start_row][start_col] != "":
                        if T[start_row][start_col] == '\\':
                            aln1 += self.s1[start_row - 1]
                            aln2 += self.s2[start_col - 1]
                            start_row -= 1
                            start_col -= 1
                        elif T[start_row][start_col] == '-':  # left
                            aln1 += '-'
                            aln2 += self.s2[start_col - 1]
                            start_col -= 1
                        else:
                            aln1 += self.s1[start_row - 1]
                            aln2 += '-'
                            start_row -= 1
                            alignments.append(AlnSeq(aln1[::-1], aln2
                                [::-1], score))
        return alignments


if __name__ == '__main__':
    my_obj = Seq("IFYNAVWSA", "IFYNAVWSAWWWVYYHAIWPG", int(argv[1]))
    submat = SubstitutionMatrix('PAM250.txt')
    alignments = my_obj.smith_waterman(submat, -5)
    for aln in alignments:
        print(aln)
```

Explanation

- The `smith_waterman` method iterates over the score matrix `F`.
- It identifies any cell with value $\geq$ threshold.
- From each such cell, it performs a standard traceback using matrix `T`.
- Every resulting local alignment is saved in the `alignments` list as an `AlnSeq` object.
- The `main` function allows the threshold to be passed as a command-line argument.

Output

Multiple local alignments are printed, each with its sequences and respective score.

20. Final Recap: Complete Summary of Sequence Alignment Practice

Algorithms Implemented

- **Needleman-Wunsch (NW)**: global alignment – aligns the entire sequences end-to-end.
- **Smith-Waterman (SW)**: local alignment – finds best matching subsequences.

- **SW with Threshold**: finds all local alignments with score $\geq$ threshold.
- **Unified NW/SW**: a flexible method that takes "nw" or "sw" as a parameter and adapts behavior accordingly.

Programming Structures

- **Classes used:**
  - `SubstitutionMatrix`: loads and queries matrices like PAM250.
  - `SequencePair / SeqPair`: stores two sequences and provides alignment methods.
  - `AlignedSequences / AlnSeq`: stores and prints alignment results.
- **Python concepts applied:**
  - Object-Oriented Programming (OOP) – inheritance, methods, encapsulation.
  - Tuple unpacking for traceback directions and score prioritization.
  - Dictionary of dictionaries for matrix lookup.

Matrix Setup and Traceback

- `F[i][j]` $\rightarrow$ alignment score between prefixes `seq2[:i]` and `seq1[:j]`.
- `T[i][j]` $\rightarrow$ direction to backtrack:
  - `'d'` = diagonal (match/mismatch)
  - `'l'` = left (gap in seq2)
  - `'u'` = up (gap in seq1)
- Gaps are introduced based on direction during traceback.

Biological Relevance

- **Substitution matrices** (e.g., PAM, BLOSUM) model evolutionary or empirical mutation likelihoods.
- **Global alignment** suits full gene comparisons or highly similar sequences.
- **Local alignment** finds domains, motifs, or conserved regions.
- **Threshold filtering** helps discard biologically irrelevant local alignments.

Test Cases and Use

- Each class and method was tested via 'if name == 'main'‘.
- Sequences and matrices were provided through '.fasta‘ and '.txt‘ files.
- Thresholds and gap penalties were customizable for different tasks.

Conclusion

This practice provided hands-on experience with core bioinformatics techniques for sequence comparison, including the construction and traceback of scoring matrices, use of substitution schemes, and flexible code design with reusable class-based Python.

...

21. Lanese Exam - Exercise C: Local Minima and Maxima Detection

Objective

This exercise evaluates the ability to:

- Parse command-line arguments using `sys.argv`,
- Read numerical data from a text file line by line,
- Identify local minima and maxima in a list of integers,
- Write the results to output files depending on user input (MIN, MAX, BOTH),
- Use error handling with `try-except`.

Exercise Description

Write a Python program that accepts two command-line parameters: a filename and an option (MIN/MAX/BOTH). The input file contains space-separated integers.

For each line:

- If option is MIN or BOTH: extract local minima and write them to `min.txt`
- If option is MAX or BOTH: extract local maxima and write them to `max.txt`

Definitions:

- A local minimum is a number strictly smaller than its neighbors.
- A local maximum is strictly greater than its neighbors.

Provided Code (Corrected)

```python
from sys import argv

def main(input_file, outputmin_file, outputmax_file, option):
    for line in input_file:
        line = line.strip().split()
        max_list = []
        min_list = []

        # First element check
        if len(line) > 1:
            if int(line[0]) > int(line[1]):
                max_list.append(line[0])
            elif int(line[0]) < int(line[1]):
                min_list.append(line[0])

        # Central elements
        for pos in range(1, len(line) - 1):
            if int(line[pos]) > int(line[pos - 1]) and int(line[pos]) > int(line[pos + 1]):
                max_list.append(line[pos])
            elif int(line[pos]) < int(line[pos - 1]) and int(line[pos]) < int(line[pos + 1]):
                min_list.append(line[pos])

```

```python
        # Last element check
        if len(line) > 1:
            if int(line[-1]) > int(line[-2]):
                max_list.append(line[-1])
            elif int(line[-1]) < int(line[-2]):
                min_list.append(line[-1])

        # Convert list of numbers to space-separated string
        max_line = ' '.join(max_list)
        min_line = ' '.join(min_list)

        # Option check
        if option == 'MIN':
            outputmin_file.write(min_line + '\n')
        elif option == 'MAX':
            outputmax_file.write(max_line + '\n')
        elif option == 'BOTH':
            outputmin_file.write(min_line + '\n')
            outputmax_file.write(max_line + '\n')

def file():
    try:
        option = input('Enter option (MIN, MAX, BOTH): ')
        input_file = open(argv[1], 'r')
        output_min = open(argv[2], 'w')
        output_max = open(argv[3], 'w')
        main(input_file, output_min, output_max, option)
    except FileNotFoundError:
        print('File not found.')
    except IndexError:
        print('Not enough arguments.')

file()
```

Listing 5: exam25.02.py - Find local minima and maxima

Python Concepts in This Script

- `from sys import argv`: used to access command-line arguments.
- `input()`: read option from user.
- `open(filename, 'r')`: read from file line by line.
- `split()`: turns a line of text into a list of numbers.
- `for pos in range(1, len(line) - 1)`: used to access all middle elements (skipping first and last).
- `append()`: to build lists of min and max values.
- `join()`: rebuilds a string from list.
- `try/except`: used for error handling (missing file or arguments).

Theory Recap

- **Lists:** mutable collections of values, accessible by index (e.g., `line[0]`, `line[-1]`).

- `if/elif/else`: conditional logic to compare numeric values.
- `for` **loop**: used for definite iteration over indices.
- **File reading**: use `open(...)` and loop through `for line in file`
- **File writing**: use `file.write(...)` and `\n` to add newlines.
- **Command-line interface**: `argv[1]`, `argv[2]`, etc. used to retrieve filenames passed when launching the script.

Example Usage

```
python3 exam25.02.py data.txt min.txt max.txt
Enter option (MIN, MAX, BOTH): BOTH
```

`data.txt` content:

```
1 2 1 1 5 3
4 2 6 1 4 5 9 1
```

`min.txt` result:

```
1 3
2 1 1
```

`max.txt` result:

```
2 5
4 6 9
```

...(contenuto precedente invariato)...

22. - Exercise A (Pairs of Integers Above Threshold)

Code

```python
from sys import argv

def count_pairs_above_threshold(numbers, threshold):
    count = 0
    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] * numbers[j] > threshold:
                count += 1
    return count

def pairs_integers():
    if len(argv) != 3:
        print("Uso: python script.py output_file threshold")
        exit(1)

    output_file = argv[1]
    try:
        threshold = int(argv[2])
    except ValueError:
        print("Errore: la soglia deve essere un numero intero.")
        exit(1)
```

```python

    try:
        writer = open(output_file, 'a')
        while True:
            line = input("Inserisci numeri separati da virgola (oppure premi invio per terminare): ").strip()
            if not line:
                break

            numbers = []
            elements = line.split(',')
            for num in elements:
                num = num.strip()
                try:
                    numbers.append(int(num))
                except ValueError:
                    print("Errore: Inserisci solo numeri separati da virgola.")
                    numbers = []
                    break

            if numbers:
                count = count_pairs_above_threshold(numbers, threshold)
                writer.write(str(count) + '\n')
                print("Coppie con prodotto sopra", threshold, ":", count)

        writer.close()
        print("Risultati aggiunti al file", output_file)

    except Exception as e:
        print("Errore durante l'elaborazione:", str(e))

pairs_integers()
```

Key Concepts

- **argv**: to receive filename and threshold.
- **Nested loops**: compute all unique pairs.
- **File writing**: appends results.
- **Try/except**: validates input and handles errors.

Sample Input/Output

```
threshold = 18
Input: 3,4,5   → Output: 1
Input: 4,4,23,1 → Output: 3
Input: -3,8,1   → Output: 0
```

23. - Exercise B (Product of All Integers)

Code

```python
from sys import argv

def compute_product():
    if len(argv) != 3:
        print("Uso: python script.py input_file output_file")
        exit(1)

    input_file = argv[1]
    output_file = argv[2]

    try:
        reader = open(input_file, 'r')
        writer = open(output_file, 'w')

        for line in reader:
            line = line.strip()
            if not line:
                continue
            try:
                numbers = [int(x.strip()) for x in line.split(',')]
                product = 1
                for n in numbers:
                    product *= n
                writer.write(str(product) + '\n')
            except ValueError:
                writer.write('Error\n')

        reader.close()
        writer.close()

    except Exception as e:
        print("Errore durante l'elaborazione:", str(e))

compute_product()
```

Concepts

- `split()` → transforms a comma-separated string to list
- `int()` → type casting
- `try/except` → catches invalid input like "a,b,c"

Concepts

- Loop from 1 to len-2 → internal values checked only
- Conditions for local max/min using neighbors
- Result list built dynamically

23. Theoretical Recap and Common Pitfalls

Python Fundamentals

- `input()` vs `argv`: first is interactive, second is for script parameters.

- `try/except`: crucial to handle invalid input or file errors.
- `open(...)`: always remember to close file or use `with` statement.
- `split()` on "," → turns strings to list.
- Looping: `for i in range(...)`, or directly `for x in list`

Common Mistakes

- Using `sys` without importing it.
- Not handling invalid conversions (`int("a")`).
- Writing to file before validating input.
- Off-by-one errors in loops (especially extrema detection).
- Wrong file mode (e.g., `'w'` vs `'a'`).

To Remember

- Always check `argv` length before using.
- Validate all inputs before computation.
- Print helpful error messages.
- Use list comprehensions where useful, but fall back to loops when clarity helps.

Teoria pratica: strumenti essenziali di Python per la programmazione imperativa (esercizi A, B, C)

1. Lettura dei dati: input e `argv`

Negli esercizi, i dati vengono acquisiti tramite due metodi fondamentali:

- `input()`: usato per leggere da tastiera (vedi esercizio A). È utile per chiedere all'utente una stringa, numero, o carattere. L'output è sempre una stringa, quindi spesso è necessario usare `int()`, `float()` per convertirla.
- `sys.argv`: usato nell'esercizio C per leggere parametri direttamente dal terminale quando si esegue il file (`python script.py valore`). È utile per automatizzare e testare.

2. Stringhe e Liste

Stringhe (`str`) sono sequenze immutabili di caratteri. Puoi accedere con gli indici: `s[0]`, `s[-1]`, e usare metodi come `s.split()`, `s.upper()`.

Liste (`list`) sono mutabili e possono contenere qualsiasi tipo di oggetto. Puoi usare:

- slicing: `lst[1:4]`, `lst[::-1]`
- metodi: `append()`, `remove()`, `sort()`

Esempi concreti:

- `split()` trasforma una stringa in lista di parole (`"A T C G".split()`)
- `list(s)` converte una stringa in lista di caratteri

3. Cicli `for` e `while`

- `for` è ideale per iterazioni note, su indici o elementi (vedi esercizi A, B).
- `while` è usato per condizioni aperte (vedi esercizio C, ricerca del massimo locale): si usa quando non si sa quante volte servirà ripetere un'azione.

Esempio:

```
i = 0
while i < len(lista):
    ...
    i += 1
```

4. Condizioni `if`, `elif`, `else`

Tutte e tre le forme sono usate negli esercizi:

- per verificare uguaglianza: `if a == b`
- per confronti: `if x > y`
- per verificare appartenenze: `if x in lista`

5. Eccezioni `try/except`

Per prevenire errori in input o operazioni illegali (come `int("ciao")`), si usa:

```
try:
    x = int(input("Insert number: "))
except ValueError:
    print("Not a number!")
```

Questa struttura è usata per garantire robustezza, ma non era esplicitamente presente nei tre esercizi. Tuttavia, è fondamentale da saper usare all'esame.

6. File: lettura e scrittura

Nel modulo C viene richiesto di leggere da file usando:

- `open("file.txt")` per aprire un file
- `read()`, `readlines()`, o iterazione con `for riga in file`
- `with open(...) as f` è la forma migliore: chiude automaticamente il file

Scrittura:

```
with open("output.txt", "w") as f:
    f.write("Hello\n")
```

7. Errori comuni da evitare

- confondere `seq[i]` con `seq(i)` (solo le funzioni si usano con parentesi tonde)
- dimenticare `int()` quando si confronta un input numerico
- usare `=` invece di `==` nei confronti
- modificare una lista mentre si itera su di essa
- dimenticare `i += 1` nei `while`

8. Buone pratiche

- Separare chiaramente logica e input/output
- Controllare l'input con `try/except`
- Usare `enumerate()` e `range(len(lista))` se servono indici
- Commentare bene ogni blocco

9. Collegamenti con esercizi d'esame

- Esercizio A: uso di `split()`, `input()`, cicli `for`, accesso a stringhe/liste
- Esercizio B: uso di `readlines()`, `float()`, elaborazione e confronto di numeri
- Esercizio C: massimo e minimo locali → analisi con `while`, `if`, `file`, `argv`

10. Conclusione

Tutti gli esercizi ruotano attorno alla capacità di manipolare dati strutturati (stringhe, liste, numeri) in modo robusto e controllato. L'uso corretto di cicli, condizioni e funzioni permette di risolvere qualsiasi problema proposto nel modulo Lanese.

Prepared by Martina Castellucci – 2025 Bioinformatics Practice Recap